YOSYS-AGENT

A multi-agent optimiser for the logic-synthesis stage of OpenROAD-flow-scripts — Yosys 0.33 + ABC, Nangate45.

What it does

The agent treats synthesis quality as a closed-loop search problem. It profiles a design, retrieves what it learned on earlier designs, proposes batches of Yosys/ABC configurations with an LLM strategist, runs them in parallel, scores them against a baseline it measures itself, and writes back what worked. **The RTL is never modified** — the entire action space is flow configuration, so correctness stays the tools' responsibility and every candidate is a legitimate implementation of the same design.

Scoping the loop to the **synthesis stage alone** is what makes it cheap: a trial costs 2 s to 4 min instead of the hours a full place-and-route iteration would need, so the agent can afford 40 real evaluations per design.

Headline result

Designs improved over baseline	5 / 5
Geometric-mean objective score	0.4424 (baseline = 1.0000)
Best single design	bp_fe_top — score 0.2709
Delay reduction range	49.5% – 95.1%
Total synthesis runs	179
Total agent wall-clock	78 min on 4 cores
Total LLM cost	$1.34

The objective

```
score = w_delay × (delay / delay_base) + w_area × (area / area_base)
```

Lower is better. The baseline substitutes into its own formula as w_delay + w_area = 1.0000, so it scores exactly 1.000 by construction and every agent score reads directly as a fraction of baseline cost. Weights are chosen per design by the profiler: 0.75 / 0.25 for gcd, aes, jpeg and bp_fe_top, and 0.60 / 0.40 for the more area-sensitive dynamic_node. Baseline and candidates run through the identical flow with the identical metric extraction, so the two columns are directly comparable.

Agentic architecture

Five roles. Four are LLM agents with a single narrow job each; the orchestrator is deterministic code, because every decision it makes is mechanical and an LLM there would only add cost and nondeterminism.

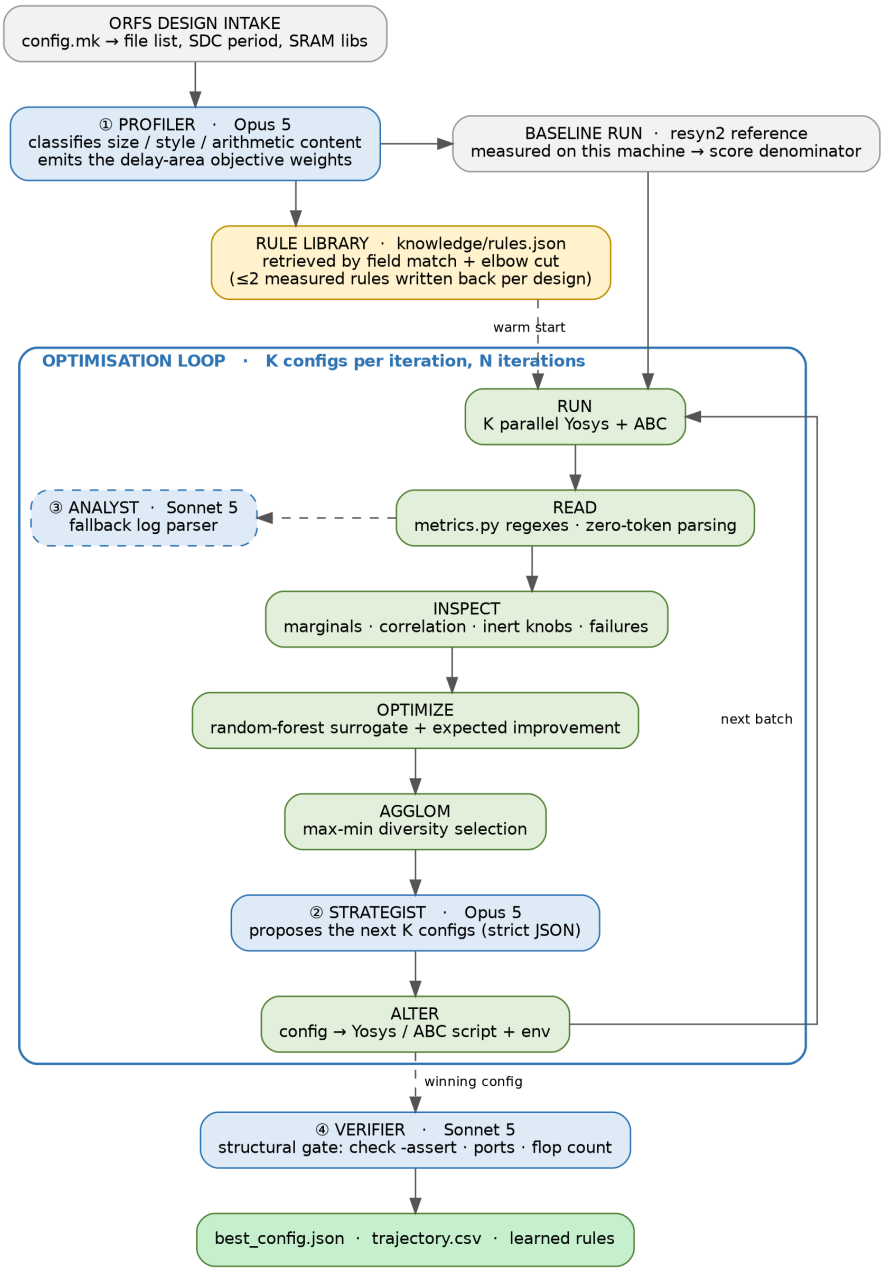


Figure 1 — Agent topology and control flow. Blue: LLM agents. Green: deterministic harness stages. Dashed: knowledge and fallback paths.

The subagents

Agent	Model	Calls /design	Responsibility
PROFILER	Opus 5	1	Classifies the design (size class, datapath vs control vs memory, arithmetic content) and proposes the delay/area weights. Runs once, before any tuning.
STRATEGIST	Opus 5	1 / iter	The hot path. Reads INSPECT summaries, prior rules and the surrogate's candidates, then returns exactly K configurations as strict JSON — half exploiting the leader, the rest probing untried corners.
VERIFIER	Sonnet 5	1	Gates the winner. Confirms check -assert passed, the port list and flop count match the baseline, and that the metric movement is consistent with a real remapping.
ANALYST	Sonnet 5	0 (fallback)	Recovers metrics if the Python log regexes ever miss. The hot loop parses logs in pure Python, so this normally costs nothing.
ORCHESTRATOR	code	—	Owns the loop, the budgets, the baseline, parallelism, early stopping and rule write-back. Deterministic by design.

Why it gets cheaper as it goes

Three mechanisms keep token cost flat while coverage grows. **INSPECT summaries** mean the strategist sees marginals, knob/score correlations, provably inert knobs and failure rates — never raw history rows — so the prompt does not grow with the search. **A random-forest surrogate with expected improvement** ranks 2,000 sampled configurations per iteration for free, and the LLM only arbitrates the shortlist. **The rule library** carries measured knob deltas across designs: it grew from 4 seeded hypotheses to 10 measured rules over this sweep, and the last design entered with 5 relevant rules already retrieved.

The tool under optimisation

The agent owns a parameterised Yosys script end to end, so it controls the coarse-optimisation stage, adder mapping and the full ABC invocation. Metrics come out of the same single Yosys process, so measurement adds no extra passes.

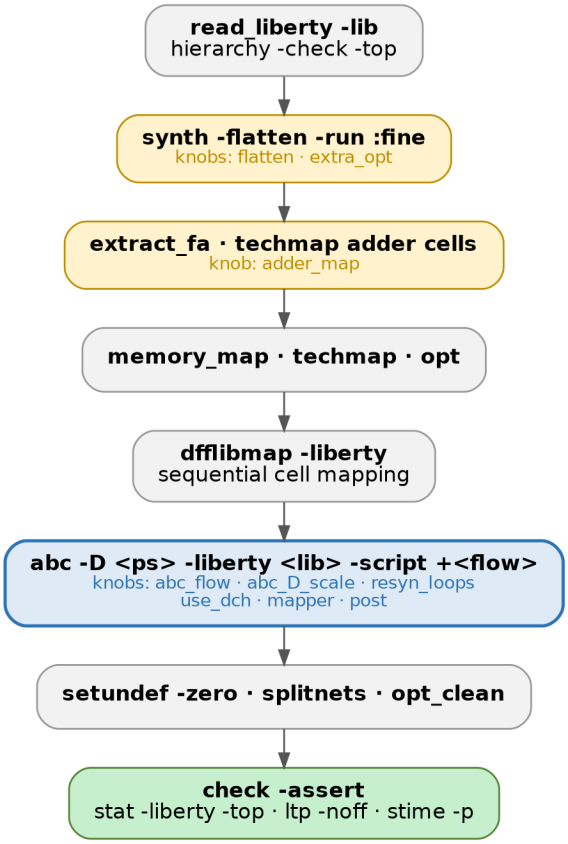


Figure 2 — Yosys flow with the agent's injection points. Amber: Yosys-stage knobs. Blue: the ABC invocation and its parameters.

ABC parameters searched

Parameter	Values	What it controls
`abc_flow`	`resyn2 | dch_map | syn2_if`	Script family. **resyn2** = balance/rewrite/refactor AIG restructuring; **dch_map** = choice computation then mapping; **syn2_if** = &syn2 / &if -g -K 6 / &synch2 / &nf refinement in ABC's GIA space.
`abc_D_scale`	`0.50 – 1.10`	Multiplies the design's SDC period to set ABC's **-D** delay target. Tightening it below 1.0 buys real delay.
`resyn_loops`	`1 – 4`	Repetitions of the chosen flow's core.
`use_dch`	`on | off`	&dch -f / dch -f don't-care-based choice computation before mapping.
`mapper`	`&nf | map | map -a | map -B 0.9`	Delay-driven (&nf, GIA space) versus classic and area-biased mapping.
`post`	`buffer;upsize;dnsize | topo;dnsize | …`	Post-mapping gate sizing and buffering.

Measurement, in-process

`stat -liberty -top` gives hierarchy-aggregated cell counts and chip area; ABC's `stime -p` gives the post-mapping critical path; `ltp -noff`, run after removing sequential cells, gives true combinational depth as an independent cross-check on delay; `check -assert` guards structural integrity. All four come from one invocation and are parsed in pure Python.

Results

Baseline is the validated `resyn2` configuration, re-measured in-harness so baseline and candidates are produced by the identical flow. Negative is better.

Design	Delay (ps) base → opt	Δ	Area (μm²) base → opt	Δ	Cells base → opt	Score	Δ score
gcd	644 → 325	**-49.5%**	460 → 471	+2.4%	362 → 330	**0.6345**	**-36.6%**
aes	1,982 → 653	**-67.0%**	14,086 → 14,484	+2.8%	12,086 → 12,162	**0.5043**	**-49.6%**
jpeg	9,312 → 1,130	**-87.9%**	91,929 → 92,001	+0.1%	74,546 → 65,521	**0.3412**	**-65.9%**
dynamic_node	2,622 → 768	**-70.7%**	20,836 → 20,709	-0.6%	11,355 → 9,178	**0.5732**	**-42.7%**
bp_fe_top	32,445 → 1,588	**-95.1%**	49,989 → 46,835	-6.3%	35,217 → 27,217	**0.2709**	**-72.9%**

Table 1 — Objective improvement per design. Flop counts are identical between baseline and optimised netlist in every design.

Runtime and cost

Design	Synth runs	Iters	Agent wall-clock	Baseline synth	LLM calls	LLM cost (USD)
gcd	41	5	1.7 min	1.2 s	6	$0.303
aes	41	5	9.4 min	26.6 s	6	$0.303
jpeg	23	2	32.1 min	137.8 s	3	$0.125
dynamic_node	33	4	5.2 min	15.2 s	6	$0.303
bp_fe_top	41	5	30.1 min	81.5 s	6	$0.303
TOTAL	**179**	**—**	**78 min**	**—**	**27**	**$1.337**

Table 2 — Search effort and spend. Wall-clock is end-to-end per design on 4 cores, including the baseline measurement and every candidate. Per-design cost is the measured sweep total apportioned by each design's LLM calls, priced at Opus 5 and Sonnet 5 rates; a standalone re-run of dynamic_node measured $0.31 independently.

What the agent discovered

On gcd and aes the winning configurations keep the baseline's own `resyn2` script and win by running it twice under a tightened delay target — a result a human sweep would be unlikely to guess. On bp_fe_top it switched to the `syn2_if` flow and cut combinational depth from 385 levels to 35, confirmed independently by the `ltp` cross-check. Across the sweep it learned and recorded 10 reusable rules, so each additional design starts warmer than the last.